Assignment Title	Software Testing and Quality Evaluation of the SE3090 Integrated System
Learning outcomes covered	LO2: Apply suitable frameworks and tools to build web, mobile, and full-stack software applications efficiently and effectively. LO3: Use best practices for integrating frameworks, managing collaborative development, applying CI/CD, ensuring code quality, and deploying software solutions.
Assignment Mode	Group assignment with individual viva
Maximum Marks	100 Marks
Contribution to the Final Grade	15%
Date published	19th September 2026
Deadline for submissions	5th October 2026
Mode of Submission	Submit through the official Learning Management System (CourseWeb).

Assignment Description

This assignment is directly connected to the SE3090 Main Assignment. You must test the same integrated system developed by your group. You are not required to build a separate application for this assessment.

The purpose is to show that your system has been tested in a planned and professional way. You must select suitable testing areas, use appropriate testing tools or frameworks, execute the tests, record the results, identify defects, and provide clear evidence of your testing work.

1. Relationship to the SE3090 Main Assignment

- Use the same ASP.NET Core Web API, PostgreSQL database, React web application, Flutter mobile application and Agentic AI subsystem developed for the SE3090 Main Assignment.
- Test the actual features and workflows implemented by your group.
- At least one test must cover a complete integrated workflow across the relevant components of the system.
- The testing evidence produced here can also support the testing-related documentation required for the SE3090 Main Assignment.

2. Testing Scope

Your group must perform suitable testing from the areas below. For each selected technical testing area, you must use an appropriate testing tool or framework. **Manual observation alone is not sufficient.**

Testing Area	What to Test	Suggested Tools / Frameworks
Backend / API Testing	Unit testing; service/business-logic testing; validation testing; controller testing; authentication and authorization testing; API integration testing.	xUnit, NUnit, MSTest, Moq, WebApplicationFactory, Postman/Newman
Database Testing	Database integration testing; constraint testing; relationship and data-integrity testing; migration testing; transaction testing.	xUnit + PostgreSQL, Testcontainers for .NET, Entity Framework Core
React Web Application Testing	Component testing; form-validation testing; protected-route testing; API-integration testing; UI-state and error-state testing.	Vitest/Jest, React Testing Library, MSW, Playwright

Flutter Mobile Application Testing	Unit testing; widget testing; form-validation testing; navigation testing; API-integration testing.	flutter_test, integration_test, mocktail/Mockito
Integration / End-to-End Testing	API integration testing; cross-component integration testing; complete business-workflow testing; cross-platform workflow testing.	Playwright, Postman/Newman, Flutter integration_test, or another justified E2E tool
Non-Functional Testing	Performance testing; load testing; stress testing; security testing; usability testing; accessibility testing; compatibility testing; reliability testing; recovery testing.	k6, Apache JMeter, OWASP ZAP, Lighthouse, axe DevTools, Playwright, and other appropriate monitoring/testing tools
Agentic AI Testing & Evaluation	Task-completion testing; agent-selection testing; tool-selection testing; structured-output validation; business-rule compliance testing; prompt-injection testing; approval-enforcement testing; failure-recovery testing; safe-failure testing.	xUnit/pytest, promptfoo, DeepEval, schema validation, deterministic test cases

Non-functional testing requirement: Performance and security testing are required. Select additional non-functional testing types where relevant to your system and justify your selection.

3. What You Need to Do

1. Identify the important features, workflows and quality risks in your SE3090 system.
2. Prepare a test plan showing what will be tested, the testing type, expected result, tool/framework and responsible member.
3. Design meaningful test cases, including normal, invalid, boundary/edge and failure cases where relevant.
4. Use suitable testing tools/frameworks and execute the tests.
5. Record actual results and clearly mark each test as Passed or Failed.
6. Record defects found, correct important defects and perform retesting.
7. Collect evidence such as automated test output, screenshots, logs, coverage, performance results, security scan results or AI evaluation results.
8. Prepare the required testing documents and submit them with the supporting evidence.
9. Prepare to explain and demonstrate your own testing contribution during the viva.

4. Testing Documents to Prepare

Document	Minimum Content
Test Plan	Scope, objectives, testing areas, tools/frameworks, test environment, responsibilities and schedule.
Test Case Document	Test case ID, feature, preconditions, steps/input, expected result, actual result and Pass/Fail status.
Defect / Bug Report	Defect ID, description, severity/priority, steps to reproduce, evidence, status and retest result.
Test Execution Summary	Tests executed, passed, failed, defects identified/fixed and a short conclusion.
Tool-Generated Evidence	Relevant automated test reports, coverage, performance results, security scans, logs or AI evaluation outputs.

5. Deliverables

- Software Testing Report (PDF) containing the test plan, testing scope, test execution summary, defect summary, and conclusion.
- Completed Test Case Document with expected result, actual result, and Pass/Fail status.
- Defect / Bug Report with retesting evidence.
- Testing tool/framework evidence, including relevant screenshots, generated reports, logs or exported results.

- Automated test source code/scripts used for the applicable testing areas.
- GitHub repository link and contribution/commit evidence related to testing.
- Any additional configuration or files required to reproduce or rerun the tests.

Important: Evidence must come from your own SE3090 system. A submission containing only theoretical descriptions of testing will not receive full marks.

6. Viva

A viva will be conducted as part of this assignment. Each student must be able to explain the tests they contributed to, the selected tool/framework, how the test was executed, what the result means, defects found, and how the system was improved. Students may also be asked to run, modify or explain a test during the viva. Individual viva performance may affect the individual mark.

7. Usage of AI

AI tools may be used to support learning, brainstorming, test-case ideas, debugging, test-script generation, documentation and code review. Students are responsible for checking, adapting and understanding all AI-assisted work. AI-generated test cases or scripts must be verified against the actual system before submission. Students must not submit testing work they cannot explain or reproduce during the viva. AI assistance must be declared according to the module requirements and the CLEAR framework.

8. Marking Scheme

Total: 100 marks | Group contribution: 40 marks | Individual contribution: 60 marks | Contribution to the final module grade: 15%

Important: This assessment is demonstration-based. Submitted documents and screenshots alone are not sufficient. Each student must personally demonstrate and explain at least one meaningful tool/framework-based testing contribution using the group's SE3090 system.

Detailed Marking Rubric

Criterion	Excellent	Good	Satisfactory	Poor	Very Poor
GROUP: Testing Strategy & Coverage (10)	Clearly explains a well-planned testing strategy. Testing scope is relevant and covers the important system components, workflows, and risks with suitable tools/frameworks.	Good strategy and coverage of most important areas with only minor gaps.	Basic strategy covers the main areas, but some important components, risks or test types are missing.	Limited testing strategy; coverage is narrow or poorly justified.	No meaningful testing strategy or coverage is demonstrated.
GROUP: Integrated & Non-Functional Testing Demonstration (15)	Successfully demonstrates meaningful integration/E2E testing and relevant non-functional testing using suitable tools. Results are clearly interpreted and connected to the actual SE3090 system.	Demonstrates integration and non-functional testing effectively with minor gaps in coverage, execution or explanation.	Basic demonstrations are completed, but coverage, tool use or interpretation is limited.	Demonstration is incomplete, mostly manual, or provides weak evidence of integrated/non-functional testing.	No meaningful integrated or non-functional testing demonstration.
GROUP: Overall Test Results, Defects & Documentation (15)	Test results are complete and traceable. Defects are clearly recorded, important fixes are shown with retesting, and required testing documents/tool evidence are complete and consistent.	Good results, defect handling and documentation with minor omissions.	Main documents and results are present, but defect/retest evidence or consistency is basic.	Documents/results are incomplete or poorly supported by actual testing evidence.	Major testing documents, results or defect evidence are missing.
INDIVIDUAL: Testing Tool / Framework Demonstration (15)	Personally and confidently demonstrates appropriate testing tool(s)/framework(s), explains why they were selected, configuration/setup, and how they are used on the actual system.	Demonstrates suitable tool/framework use with good understanding and only minor gaps.	Can run and explain basic tool/framework usage but shows limited understanding of configuration or purpose.	Limited demonstration; relies heavily on others or cannot clearly explain how the tool/framework is used.	Cannot demonstrate meaningful personal use of a testing tool/framework.

INDIVIDUAL: Test Implementation & Execution (15)	Shows meaningful personally implemented tests and executes them successfully. Test design includes appropriate normal, invalid, boundary/edge and/or failure cases and meaningful assertions/checks.	Good implementation and execution with minor gaps in test variety, assertions or coverage.	Basic tests are implemented and run, but scenarios or assertions are limited.	Few/trivial tests, weak implementation, or difficulty executing own tests.	Cannot show or execute meaningful personally implemented tests.
INDIVIDUAL: Results, Defects & Retesting (10)	Clearly interprets own test results, identifies meaningful defects/issues, explains their cause/fix where applicable, and demonstrates retesting or verification.	Good understanding of results and defect handling with minor gaps.	Can explain basic results and some defect/retest evidence, but analysis is limited.	Results are shown with little interpretation or weak defect/retest evidence.	Cannot explain test results or provide meaningful defect/retesting evidence.
INDIVIDUAL: Technical Contribution (5)	Clear individual ownership is visible through test code/scripts, Git history and related fixes/evidence; contribution is consistent and traceable.	Good identifiable contribution with minor gaps in traceability.	Some identifiable contribution exists, but ownership/evidence is limited.	Very limited or unclear individual contribution.	No identifiable individual testing contribution.
INDIVIDUAL: Viva & Technical Understanding (15)	Demonstrates strong understanding of own testing approach, code/scripts, tools, results and related system behaviour; confidently answers questions and can run, explain, modify or troubleshoot a test when requested.	Good understanding and demonstration with minor difficulty on advanced questions or modifications.	Basic understanding, but has difficulty explaining some technical decisions, results or test changes.	Weak understanding and significant difficulty explaining or modifying submitted testing work.	Cannot explain, reproduce, modify or demonstrate submitted testing work.